
Learning Bault

A complete study plan for the codebase — read in the order that makes each next file cheaper than the last.

62,038 lines of TypeScript across 287 files · 15 API modules · 44 tables · 27 migrations · 55 test files
DIVE1.md: 42 parts, ~22,000 lines · Generated 16 September 2026

Before you start: the four rules that halve the time

This codebase has an unusual property — it explains itself twice. Once in `DIVE1.md`, at length, and once in the doc comments sitting next to the rules they describe. Most of your speed comes from knowing which of the two to read at any moment.

1. Read DIVE1 backwards.

DIVE1 says it about itself: *later parts win*. Parts 1–8 describe the system as first built; Parts 9–42 are changelogs for everything that replaced it. Reading forwards means spending hours on bin `capacity`, `BIN-A-001` naming, two-tab receiving and quantity-based intake — all of which were deleted. This is the single biggest waste of time available here.

2. Navigate by heading, never by line number.

DIVE1 grows with every change, so line numbers go stale within a day. Search for `# Part 24 including the trailing space`, so Part 2 does not match Part 24.

```
grep -n "^# Part" DIVE1.md # the 42 parts and where they start
awk '/^# Part 24 /,/^# Part 25 /' DIVE1.md | grep "^## " # one part's sections
```

3. When the prose gets tedious, open the file it describes.

The code carries the same reasoning at the point where each rule is enforced, and it cannot drift, because it sits next to the rule. A paragraph of DIVE1 you are re-reading is a file you should be reading instead.

4. Read the test names as the specification.

Integration test names are sentences. Reading only the `it(...)` lines of a file tells you what that module promises, in less time than the module takes to read.

```
grep -h "it(" tests/integration/shp-outbound.test.ts
```

The one idea everything else hangs off

The database, not the shelf, is the authority on who owns each item, where it is, and what money is owed.

Because the platform is the sole system of record for ownership, custody and money, the code is built around a small set of hard invariants — several enforced by database triggers, so that no application bug can violate them:

Invariant	Where it is enforced
Item records are never deleted	<code>db/sql/0001_append_only.sql</code> — a reject-mutation trigger
Every item has exactly one owner, always	<code>cst/custody.service.ts</code> — the only code allowed to move an item
Every ownership / location / state change leaves an immutable custody trail	Custody events are append-only, by trigger
The money ledger is append-only	Same trigger family; <code>pay/ledger.service.ts</code>
A wallet balance is never stored — it is the sum of its ledger rows	<code>pay/wallet.service.ts</code> ; the <code>property</code> test suite
Every state-changing action is written to an immutable audit log	<code>sec/</code>
A price is a dated rule; the rule in force at charge time is frozen onto the charge	<code>prc/pricing.service.ts</code> — the price-freeze snapshot

If you internalise nothing else on day one, internalise this table. Almost every design decision in the system is downstream of it, and once you stop re-deriving these principles in each new context, the rest of the document reads at roughly double speed.

The shape of what you are learning

Module	Owns	Files	Lines	Start here
cst	Custody kernel — the only code that moves an item	9	1,399	<code>lifecycle.ts</code> , <code>custody.service.ts</code>
inv	Intake, taxonomy, parcels, labels	16	3,115	<code>item-classes.ts</code> , <code>intake.service.ts</code>
shp	Outbound: carriers, boxes, quotes, customs, fulfilment	19	4,573	<code>carriers.ts</code> , <code>shipment.service.ts</code>
dis	Services on a card: grading, inspection, deslab, consignment	17	3,154	<code>service.service.ts</code>
pay	Ledger, wallet, checkout, top-ups, withdrawals, chargebacks	14	2,503	<code>ledger.service.ts</code>
mkt	Marketplace, offers, swaps, escrow purchases, the Bault store	14	2,053	<code>purchase.service.ts</code>
acc	Accounts, sessions, passwords, addresses	15	1,332	<code>session.service.ts</code>
esc	Escrow on a private deal	5	1,234	Part 22
vlt	The customer's view of their own items	5	924	—
adm	Admin surface, shelf yield	5	903	Part 29
not	Event catalogue, transactional outbox, email	9	696	<code>event-types.ts</code>
sup	Helpdesk, append-only ticket threads	4	525	Part 17
prc	Effective-dated pricing rules, the public price list	5	394	<code>pricing.service.ts</code>
sec	RBAC, audit, PII	9	239	—
med	Media / photographs	3	195	—

Web app	What is in it	Files	Lines
<code>areas/customer</code>	Vault, marketplace, wallet, shipping, support, help, auth, marketing	34	13,133
<code>areas/warehouse</code>	Receiving bench, intake bench, stow, dispatch	8	3,620
<code>areas/admin</code>	Console, shelf yield, house store, wallet requests	5	2,229
<code>shared/ + index.css</code>	i18n, api client, routing, primitives, the design system	—	~6,600 CSS

The plan — ten days, about 24 hours

Each day is roughly 2–3 hours. **Stop at the end of any day and you still have a coherent mental model** — the plan is ordered so that nothing later is a prerequisite for something earlier. Every day ends with a task you can only complete if you actually understood the reading, because recognition is not knowledge and only the exercise tells the two apart.

Phase 1 · The spine (days 1–2)

Afterwards you can reason correctly about any part of the system, and recognise a proposed change that would break an invariant.

DAY 01 The thesis, the map, and where it stands now

2h 30m

You are not learning code today. You are learning what this system refuses to do.

Read	DIVE1 opening paragraphs, above "How this document is organized" (5 min) · the contents list itself, all 42 bullets (15 min) · then Parts 42 → 39 , backwards , in full (70 min).
Then	<code>docs/design/03-intake.md</code> — the intake path in the order the work happens (20 min).
Open	<code>apps/api/src/db/sql/0001_append_only.sql</code> and read every trigger. This is the constitution, as code (20 min).
Exercise	Write the invariant table on page 2 from memory. Then run <code>pnpm db:reset</code> and <code>pnpm test:web</code> so you know the loop works before you need it.
Self-test	Why is a wallet balance never stored? Why can nothing in the product delete a custody event?

DAY 02 The custody kernel

2h 30m

Every item movement in the product goes through one service. Read it until you could rewrite it.

Read	DIVE1 Part 4 — Custody, Intake & Vault — every word (50 min).
Open	<code>cst/lifecycle.ts</code> (every legal state change, in ~30 lines) → <code>cst/custody.service.ts</code> → <code>cst/stow.service.ts</code> (why there is no bin capacity) → <code>inv/item-classes.ts</code> (the closed taxonomy and the lot rule).
Then	Parts 37 , 34 , 32 , 24 , backwards — the four passes that rebuilt receiving and stow (45 min).
Exercise	Draw the lifecycle state machine on paper from <code>lifecycle.ts</code> . Then find, in the code, the three places that read <code>bin.active</code> and the one that does not. (Part 37 tells you if you are stuck.)
Self-test	Why is there no bin capacity? Why is a serial minted, never typed? What is the difference between a parcel and an item?

Phase 2 · Working knowledge (days 3–6)

Afterwards you can change code in any module without breaking an invariant.

DAY 03 Money: the ledger, the wallet, the price

2h 30m

Money is the half of this system with the least tolerance for a misunderstanding.

- Read DIVE1 **Part 5** at section level (PRC / PAY / MKT), then Parts **30, 23, 16, 13** backwards (90 min).
- Open `pay/ledger.service.ts` → `pay/wallet.service.ts` → `prc/pricing.service.ts` (the effective-dated resolve, and the snapshot that freezes a price) → `pay/money-terms.ts` → `shared/money.ts`.
- Exercise Trace one storage charge end to end: the rule in the seed → the worker sweep that bills it → the ledger rows → the balance the customer sees. Name every file it passes through.
- Self-test If an admin changes the intake fee today, what happens to a charge raised last week, and which line of code guarantees it?

DAY 04 Selling, trading, escrow

2h 15m

Four engines that were complete before anything could reach them.

- Read Parts **41, 31, 30, 22, 18**, backwards (80 min).
- Open `mkt/purchase.service.ts` (the atomic purchase and its row lock) → `mkt/house-store.service.ts` (when an item comes into existence, and why then) → `esc/`.
- Exercise Find the rule that stops a party accepting a price they proposed themselves — in the API *and* in `SellerPanels.tsx`. Note that the control is not rendered rather than disabled, and work out why that distinction was made.
- Self-test When a card is bought from the Bault store, when does its item record come into existence, and why then rather than at stow?

DAY 05 Outbound shipping

2h 30m

The densest module in the system, and the best-documented. Read it as a worked example of how this codebase thinks.

- Read Parts **41, 40, 20** and the SHP half of **Part 6** (75 min).
- Open `shp/carriers.ts` (a service is defined by what it *refuses*) → `shp/boxes.ts` → `shp/parcel-profile.service.ts` → `shp/shipment.service.ts` (quote, priceServices, pickBest) → `shp/fulfilment.ts` → `packages/adapters/src/shipping.ts`.
- Exercise Hand-compute a quote: two graded slabs, a medium box, to Germany, insured \$2,000. Which services are eligible, which refuse and on which rule, and what is the total — carrier + handling + premium + add-ons? Then check yourself against `tests/integration/shp-outbound.test.ts`.
- Self-test Why is the chosen box stored on the shipment rather than only used in the quote? Why is rush a Bault charge and not a carrier one?

DAY 06 Services, notifications, the worker

2h

Everything that happens to a card, and everything that happens on a timer.

- Read Parts **33, 21, 19** backwards, and the DIS / NOT halves of **Part 6** (70 min).
- Open `dis/service.service.ts` → `dis/grading-tiers.ts` → `not/event-types.ts` → `not/outbox/` (the transactional outbox) → `apps/worker/src/jobs/`.
- Exercise Add nothing; answer this instead: why is the outbox a table rather than a direct send inside the transaction? Find the failure mode it exists to prevent.
- Self-test Which adapters are real right now and which are simulated? (Parts 39–40.)

Phase 3 · The surfaces (days 7–8)

DAY 07 The design system, and why it is a system

2h 15m

The CSS is the specification's implementation. **DESIGN.md** is the spec, and where they disagree the CSS is right.

- Read **DESIGN.md** front to back (40 min) — it is 354 lines and every rule in it was written against something an audit found.
- Then Parts **42, 29, 28** (45 min).
- Open **apps/web/src/index.css** — the token block only, then skim the component sections · **scripts/design-lint.mjs** · **shared/ui/primitives.tsx** · **shared/ui/Serial.tsx** (why bidi isolation is not optional) · **shared/i18n.tsx**.
- Exercise Run **node scripts/design-lint.mjs**. Then deliberately break one rule — a **margin-left**, or a hex literal outside the token block — and confirm the linter names it. Undo.
- Self-test Why is money *not* set in the mono face when serials are? Why is there exactly one shadow, and what is it for?

DAY 08 The screens, driven by hand

2h 30m

Read the UX audit parts as a list of the mistakes this product already made, so you do not make them again.

- Read Parts **31 and 30** in full — thirty defects found by using the product rather than reading it (60 min). Then Parts **12, 17, 25** (40 min).
- Open **tests/ux/*.test.tsx** — read the **it(...)** lines of all nine files. Each one pins a real defect and names it.
- Exercise Run the app (**pnpm dev**) and drive one money-moving flow by hand as a customer: top up, buy, ship. Note every place a price appears before its button, and every place it would not.
- Self-test What does a suspended user keep access to, and why that and nothing else?

Phase 4 · Depth and edges (days 9–10)

DAY 09 Security, the perimeter, and what is unfinished

2h 15m

- Read Parts **39, 27, 26, 13** backwards (75 min), then **docs/production-readiness.md** — including its explicit list of what was left undone.
- Open **acc/session.service.ts** · **acc/password.service.ts** · **sec/** · **shared/observability/health.controller.ts** · **main.ts** (headers, CORS, rate limiting, graceful shutdown).
- Exercise Part 26 found a sandbox payment adapter that settled a \$5,000 top-up against a fake token. Find the code that now prevents a sandbox adapter being selected in production, and name the file.
- Self-test What happens to existing sessions when a password is changed, and which part fixed that?

DAY 10 The scaffolding — last, or never

2h 30m

Parts 1, 2, 7 and 8 are 55% of DIVE1 and the least current part of it. Read them only now, and only at section level.

Read	Part 2 (API bootstrap, Drizzle client, migrations, the seed) properly — the seed is worth real attention, because it is the clearest statement of what a healthy database looks like. Skim Parts 1, 7, 8 by heading only.
Open	<code>db/schema/index.ts</code> · <code>db/seed.ts</code> · <code>db/migrations/*.sql</code> — each migration opens with why it exists · <code>vitest.workspace.ts</code> , which explains what each of the eight test projects defends.
Exercise	Run the whole suite from a clean seed: <code>pnpm test:seeded</code> . Then make a one-line change that should break exactly one test, and confirm it breaks that one.
Self-test	Which suites need a live database and a running API, and which do not? Why is <code>pnpm test</code> serial rather than parallel?

The shortcut: files that are the short version of DIVE1

These carry the same reasoning as the prose, at the point where each rule is enforced, so they cannot drift. When a part of DIVE1 gets tedious, open the file it describes instead.

apps/api/src/db/sql/0001_append_only.sql	the invariants, as triggers
apps/api/src/modules/cst/lifecycle.ts	every legal state change, in 30 lines
apps/api/src/modules/cst/custody.service.ts	the only code allowed to move an item
apps/api/src/modules/cst/stow.service.ts	why there is no bin capacity
apps/api/src/modules/inv/item-classes.ts	the taxonomy, and the lot rule
apps/api/src/modules/inv/parcel.schema.ts	why a parcel is not an item
apps/api/src/modules/shp/carriers.ts	a service defined by what it refuses
apps/api/src/modules/shp/shipping-options.ts	insurance, signature, add-ons, customs
apps/api/src/modules/prc/price-list.service.ts	the price list, written for a collector
apps/api/src/modules/mkt/house-store.service.ts	when an item comes into existence
apps/api/src/db/migrations/*.sql	each one opens with why it exists

By goal: what to read for the job in front of you

Once you have Phase 1, go straight to the row that matches your task and read those parts **newest first**.

You are working on	Read these parts	Then open
Intake, receiving, shelves	41, 37, 36, 34, 32, 24, 15, 14, 4	inv/intake.service.ts, cst/stow.service.ts
Marketplace, the Bault store	41, 31, 30, 22, 18, 5	mkt/purchase.service.ts, mkt/house-store.service.ts
Money, wallet, fees	30, 23, 16, 13, 5, 3	pay/ledger.service.ts, prc/
Outbound shipping	41, 40, 20, 6 (SHP)	shp/shipment.service.ts, carriers.ts, boxes.ts
Services (grading, cracking, splitting)	33, 19, 6 (DIS)	dis/service.service.ts
Screens and the design system	42, 31, 29, 28, 12, 8	DESIGN.md, apps/web/src/index.css
The public / marketing surface	42, 29	areas/customer/marketing/LandingPage.tsx, auth/AuthPage.tsx
Auth, security, perimeter	39, 27, 26, 17, 13, 3	acc/, sec/
Notifications	21, 6 (NOT)	not/event-types.ts, apps/worker/src/jobs/
Hebrew / English, copy	40, 33, 10	apps/web/src/shared/i18n.tsx

The commands you will actually use

<code>pnpm dev</code>	API first, then the web app (ordered on purpose)
<code>pnpm db:reset</code>	reseed – the 9-item Rayquaza seed is the clean state
<code>pnpm test</code>	every suite, one after another (they share one database)
<code>pnpm test:seeded</code>	migrate, reseed, then the whole suite
<code>pnpm test:web</code>	pure units – no database, no server
<code>pnpm test:ux</code>	the suite that RENDERS components (jsdom)
<code>pnpm test:integration</code>	module flows against a live API
<code>node scripts/design-lint.mjs</code>	the design system's own linter
<code>pnpm typecheck</code>	every package

Two things that will waste an afternoon if nobody tells you

- **Integration tests leave non-Rayquaza fixtures behind.** Run `pnpm db:reset` after them. The 9-item seed is the clean state.
- **A long-lived dev database drifts.** Part 41 records a shipping test failing because the worker's tracking refresh had moved a seeded shipment on. It passes from a clean reset. When a test fails strangely, reseed *before* you debug.

Self-test: you have the spine when you can answer these cold

Each maps to a part, so a blank tells you exactly where to go back to.

1. Why is there **no bin capacity**? (Part 24)
2. Why is a serial **minted, never typed**? (Parts 9, 24)
3. Why does closing an **empty parcel** refuse by default? (Parts 24, 38)
4. Why is `type_class` a **closed vocabulary**? (Part 14)
5. Why is a wallet balance **never stored**? (Part 5)
6. What is the difference between a **parcel** and an **item**? (Part 15)
7. Which adapters are **real** and which are **simulated** right now? (Parts 39–40)
8. When a card is bought from the **Bault store**, when does its item record come into existence, and why then? (Part 41)
9. Why is a chosen shipping **box stored on the shipment** rather than only used in the quote? (Parts 20, 41)
10. Why does the **landing page** fetch its prices rather than state them, and what does it show when that request fails? (Part 42)
11. Why is **storage** an included period followed by a proportion of the intake fee, rather than a daily rate? (Part 16)
12. Why can a **suspended user** still sign in? (Part 17)

Get 1–5 right and everything else reads at roughly double speed, because you stop re-deriving the same principles in each new context.

What to skip, stated plainly

- **Parts 1, 2, 7 and 8**, unless you are changing the build system or the React shell. They are 55% of the document and the least current part of it. Day 10 exists so that you read them *after* they are cheap, not instead of never.
- **Any part describing a screen you are not working on**. The UX parts (12, 28, 29, 30, 31) are excellent, but they are reference, not spine.
- **docs/dive1/part1.md ... part8.md** — earlier split copies of Parts 1–8. `DIVE1.md` is the current one.
- **Reading anything forwards**. Said three times in this document because it is the single biggest waste of time available here.

The honest caveat about "every line of code"

62,000 lines read at any useful comprehension is roughly 60–80 hours, and most of it would not survive the month — this codebase rewrites itself in passes, which is what 42 changelog parts *are*. This plan gets you to the point where **you can derive any line you have not read**, which is both faster and more durable than having read it. If you genuinely want the remaining coverage afterwards, the cheapest route is the one this repository already gives you: read the `it(...)` lines of all 55 test files (about 3 hours) and then open only the files whose promises surprised you.